YAMLess Case Study — Business Day Scheduler

YAMLess: Generating Azure Infrastructure from Code

Case Study — krogertechnology/pos-service-business-day-scheduler

---

What is YAMLess?

YAMLess is an **Infrastructure from Code (IFC)** tool. Given a code repository, it produces a complete, policy-validated Azure Terraform workspace — with no configuration files, no schema to fill in, and no IaC expertise required from the developer.

The pipeline has four stages:

```
Repo on disk
    │
    ▼
┌──────────────────────┐  Reads Dockerfile, package.json / requirements.txt /
│  1. Static Analyzer  │  pom.xml, .env files, local.settings.json, source code.
│                      │  No execution, no API calls. Produces a typed AppProfile.
└──────────┬───────────┘
           │  (fires only when any field is low/medium confidence)
           ▼
┌──────────────────────┐  Claude reads relevant source files and fills in what
│  2. LLM Gap-Filler   │  static analysis couldn't determine — via structured
│                      │  tool use only, no free-form parsing.
└──────────┬───────────┘
           │
           ▼
┌──────────────────────┐  Jinja2 root templates + static Terraform modules →
│  3. Renderer         │  ready-to-apply workspace. Outputs Container
```

```
Apps
  │                        │  or AKS, selected by --target.
  └────────────┬───────────┘
               ▼
  ┌────────────────────────┐  checkov policy-as-code. Security posture is
  │  4. Validator          │  enforced by static module files — the LLM cannot
  │                        │  affect it.
  └────────────────────────┘
```

Applying YAMLess to pos-service-business-day-scheduler

pos-service-business-day-scheduler is a Kroger NGPOS service that publishes business-day records to a downstream batch service for six US timezones (Eastern, Central, Mountain, Pacific, Hawaii, Alaska). It is a **Python 3.13 Azure Durable Functions** application: six cron-scheduled timer triggers fire at midnight in each timezone, each starting a Durable orchestrator that calls an activity function which HTTP POSTs to an internal batch service using PingOne OAuth tokens for authentication.

The service has no database, no queue, no blob storage, and no inbound HTTP endpoint — a purely outbound, schedule-driven integration.

Command run

```
yamless generate /tmp/pos-service-business-day-scheduler \
  --no-llm \
  --target aks \
  --out-dir ~/yamless-outputs/business-day-scheduler-infra \
  --validate
```

Profile detected

Field	Detected value	Confidence	Notes
Runtime	`python`	high	FROM `mcr.microsoft.com/azure-functions/python:4-python3.13`
Runtime version	`3.13`	high	`python3.13` extracted from image tag
Shape	`cron-job`	high	`azure-functions-durable` in requirements.txt
Scale trigger	`schedule`	—	Derived from cron shape
Port	`null`	—	Correct — no HTTP server, headless timer worker

Field	Detected value	Confidence	Notes
Database	`null`	—	Correct — no DB dependencies
Queue	`false`	—	Correct — no queue packages
Secrets	10 found	high	All from `local.settings.json`: OAuth credentials, PingOne OIDC config, batch service URL

Secrets detected: - `OAUTH_USERNAME`, `OAUTH_PASSWORD` — service account credentials - `PING_CLIENT_ID`, `PING_ISSUER_URI`, `PING_REDIRECT_URI`, `PING_SCOPE`, `PING_GRANT_TYPE`, `PING_RESPONSE_MODE`, `PING_RESPONSE_TYPE` — PingOne OIDC flow config - `BATCH_SERVICE_BASE_URL` — downstream service endpoint

Output generated (AKS target)

```
business-day-scheduler-infra/
├── terraform/
│   ├── main.tf              # VNet, AKS cluster, Workload Identity,
Key Vault
│   ├── variables.tf
│   ├── outputs.tf
│   ├── providers.tf
│   └── modules/
│       ├── vnet/            # VNet + AKS subnet
│       ├── aks_cluster/     # AKS with OIDC, Workload Identity, Key
Vault CSI, KEDA
│       ├── aks_workload_identity/  # Federated credential + KV
Secrets User role
│       └── keyvault/        # Key Vault with RBAC + network deny-all
└── k8s/
    ├── namespace.yaml
    ├── service-account.yaml         # Workload Identity annotation
    ├── secret-provider-class.yaml   # All 10 secrets mounted via CSI
driver
    ├── deployment.yaml              # No HTTP probes — correct for a
scheduled worker
    └── network-policy.yaml          # Default-deny; no ingress allow
rule
```

Note: `service.yaml`, `ingress.yaml`, and `hpa.yaml` are **not generated** — correct for `cron-job` shape.

43/43 policy checks passed — the output is valid and deployable.

What the output correctly captured

- Cron-job shape: no Service, Ingress, or HPA Kubernetes manifests
- All 10 runtime secrets projected through Key Vault CSI driver into the pod as environment variables
- Workload Identity for pod-level Key Vault access (no long-lived credentials)
- AKS Workload Identity with federated credential scoped to the app's Kubernetes service account

- Key Vault network deny-all with RBAC-only access
- Runtime tag `python` on all Azure resources
- All security golden standards: TLS 1.2, public network access disabled, consistent tagging

What was missed

Missing	Root cause
Kubernetes CronJob resource	The output uses a Deployment for the pod spec, but a true timer-triggered workload should use a Kubernetes `CronJob` manifest (or rely on Azure Functions' built-in KEDA timer trigger for AKS). The generated Deployment runs continuously and waits for the Durable Functions host to fire the timer — correct behaviour but not the idiomatic Kubernetes pattern.
No infrastructure for the batch service endpoint	`BATCH_SERVICE_BASE_URL` is stored in Key Vault as a secret but the downstream service itself is external — yamless correctly doesn't try to provision it.

What this test case drove into YAMLess

1. Wrong Python version from Azure Functions image tags

Root cause: The base image tag `4-python3.13` caused the existing pattern `python[:\-](\d[\d.]*)` to match `python:` and capture `4` (the Azure Functions host version) as the Python version, returning `runtime_version: "4"` instead of `"3.13"`.

Fix applied: Added `r"python(\d+\.\d+)"` (requires a decimal point — matches `python3.13` but not `python:4`) before the existing colon/dash pattern. This correctly captures `3.13` for any `X-pythonY.Z` style Azure Functions image tag.

---

2. `local.settings.json` was not scanned

Root cause: Azure Functions uses `local.settings.json` (under the `Values` key) instead of a `.env` file for local configuration. The env scanner only read `.env*` glob patterns. All 10 secrets were present in `local.settings.json` but returned `secrets: []`.

Fix applied: Added `_scan_local_settings()` to the env scanner. It reads `local.settings.json` if present, parses the JSON, and yields all uppercase keys from `Values`. Combined with the broader secret patterns below, all 10 environment variables were detected.

---

3. Secret patterns were too narrow for OAuth/OIDC/endpoint variables

Root cause: `_INFRA_SECRET_PATTERNS` was focused on infrastructure credential names (`DATABASE_URL`, `STRIPE_*`, `JWT_SECRET`, etc.) and missed the OAuth/PingOne variable naming conventions used by this service: `OAUTH_PASSWORD`, `PING_CLIENT_ID`, `BATCH_SERVICE_BASE_URL`.

Fix applied: Added three broader patterns: - `\w+_(PASSWORD|SECRET|TOKEN|KEY)$` — catches any credential by suffix - `(OAUTH|OIDC|PING|AUTH|SSO)_\w+` — catches any OAuth/OIDC/SSO config var by prefix - `\w+_(BASE_URL|ENDPOINT|URI)$` — catches external service endpoint vars

---

4. `azure-functions-durable` had no shape hint

Root cause: The `azure-functions-durable` package was not in any shape table, so the shape fell to `unknown` with `low` confidence. Without the LLM gap-filler (`--no-llm`), no shape could be inferred statically.

Fix applied: Added `azure-functions-durable` to `_CRON_PACKAGES`. Durable Functions are overwhelmingly used for timer-orchestrated or batch-processing patterns; the LLM gap-filler can correct the rare HTTP-triggered durable function case (confidence would remain `medium` for LLM correction).

Learnings for YAMLess

Azure Functions is a first-class pattern — CronJob manifest would complete the picture

The generated output is structurally correct and policy-compliant. However, the Kubernetes manifests emit a `Deployment` for a workload that is fundamentally schedule-driven. A more idiomatic output for a cron-job shape on AKS would be a `CronJob` manifest rather than a `Deployment` — or alternatively, configure KEDA's timer trigger to scale the Deployment from zero between scheduled runs.

This is a rendering concern, not an analysis concern. The shape is detected correctly; the renderer just maps all shapes to Deployment today.

Summary

This service was detected correctly end-to-end without the LLM:

- **Runtime**: Python 3.13 (from a non-standard Azure Functions image tag pattern)
- **Shape**: cron-job (from `azure-functions-durable` in requirements)
- **Secrets**: all 10 (from `local.settings.json`, previously unscanned)

The generated AKS Terraform and Kubernetes manifests are structurally correct for a headless scheduled workload. **43/43 checkov checks pass.** All secrets are projected through Key Vault → CSI driver → pod environment variables with no hardcoded credentials anywhere in the generated output.

Four static analyzer improvements were shipped as a direct result of this test: Azure Functions image version extraction, `local.settings.json` scanning, broader OAuth/endpoint secret patterns, and `azure-functions-durable` shape hint.

Generated by YAMLess — https://github.com/zafar-ahmed-kr/YAMLess